# 物理モデルの自動構築のための数式集合の検索<br>大規模言語モデルとの比較の方法と費用

報告者：M2 一洋

**Abstract**

本研究の目的は、複数の情報源から収集した数式の中から、作りたいモデルに使える可能性の高い数式を提示する手法を開発することである。提案手法は 2 段階の処理から構成される。第 1 段では、数式を、作りたいモデルの説明文と式周辺の文章の近さと、入出力変数の重複率で採点し、上位 $N_1$ 件に絞る。第 2 段では、10 個の特徴量を持つ多層パーセプトロンで並べ直し、上位 20 件を提示する。提案手法を 11,146 式と 1,731 ケースで評価した結果、上位 20 件に入った正解式の割合は、古典的な情報検索手法の 0.906 から 0.981 に上がった。本報告では、大規模言語モデルに全数式を渡して式を選ばせる比較の方法と、その費用を示す。費用の 68〜90% は、呼び出しのたびに全数式を読み出す料金である。

## 1 進捗と議論のポイント

- 大規模言語モデル（large language model, LLM）との比較の方法を決め、実行の準備を済ませた。LLM に、データベースの全 11,146 式と、テストケース 1 件のクエリ（作りたいモデルの説明文と入出力変数）を渡して式を最大 20 個選ばせ、選んだ式が正解式かを、提案手法と同じく式の ID で判定する。
- テストケース 1,673 件を 2 つのモデルで 1 回ずつ解かせる費用を見積もった（モデルの組み合わせによって 482〜780 ドル）。有料のため、まだ実行していない。
- **議論したい点**：次の 2 点について、意見をいただきたい。
  (1) この比較方法は公平か。この方法で、提案手法と LLM の性能を本当に比べられるか。
  (2) 費用の 68〜90% は、呼び出しのたびにデータベース（952,185 トークン。トークンは LLM が文字列を数える単位）を読み出す料金である。もっと費用を抑えて比べる方法はないか。

## 2 大規模言語モデルとの比較の方法

### 2.1 何を比べるか

LLM は多くの文献を学習しており、データベースを渡せば、作りたいモデルに使う式を自ら選べる可能性がある。そこで、LLM にも提案手法と同じデータベースとクエリを渡して式を選ばせ、提案手法と同じ規則で判定して、提案手法と古典的な情報検索手法（以下 baseline）と比べる。評価用の各ケースには、クエリと、作りたいモデルを構成する式の集合（正解集合。その式を正解式とよぶ）を決めてある。ケースは出典の単位で訓練用、検証用、テスト用に分け、分け方を 10 通り変えて評価する。これまでの比較では、LLM に式を書かせ、書かれた式が正解式と同じかを別の LLM に判定させていた。そのため、LLM と提案手法は別の問題を解いており、判定の方法も違っていた。10 月 14 日に加藤先生と議論し、データベースから式を選ばせて式の ID で判定する方法に改めた。

### 2.2 LLM に渡すもの

1 回の呼び出しで、LLM に、指示文、データベース、テストケース 1 件のクエリを渡す（指示文と入力の形は Appendix A）。各手法が使う情報を Table 1 にまとめる。

- **データベース**：各式を、ID、式の表記、分野ラベル（式が扱う分野の名前）の 3 つだけで渡す。使う LLM の入力の上限は 100 万トークンであり、この形でも、指示文とデータベースで 952,185 トークンになる。変数記号とその定義を加えると約 199 万トークンになって上限を超えるため、変数記号の一覧と定義、式周辺の文章は渡さない。ただし、式の表記には変数が記号で書かれている。
- **出典**：渡さない。式の並びを無作為にし、ID も 1〜11,146 の番号に振り直す（元の ID は出典の名前を含む）。提案手法は出典を使わないためである。また、合成のケース（微分方程式から始め、プログラムで式を足して作った 1,000 件）の正解集合は、同じ出典の式を優先して作ったので、出典が分かると、LLM だけが答えの手がかりを持つことになる。
- **解いた例**：渡さない。正解の分かっているケースを例として見せずに解かせる（ゼロショット）ためである。

Table 1

| 情報 | 提案手法 | baseline | LLM |
|---|---|---|---|
| クエリ（説明文、入力変数、出力変数） | ○ | ○ | ○ |
| 式の表記と分野ラベル | ○ | ○ | ○ |
| 式の変数記号の一覧 | ○ | ○ | × |
| 式周辺の文章 | ○ | ○ | × |
| 変数記号の定義の文章 | × | × | × |
| 出典 | × | × | × |
| 正解の分かっているケース | 訓練用と検証用 | 検証用 | × |

- **答え方**：作りたいモデルに使う式の ID を、可能性の高い順に最大 20 個答えさせる。モデルを構成すると判断した式を先に挙げ、20 個に満たなければ、使う可能性の高いほかの式で埋めるよう指示する。指示文には、モデルの式の組が満たす条件として、Kato and Kano [1] の要件のうち変数と式の数え方に関する 3 つ（入出力変数がすべていずれかの式に現れる、式の数が入力変数以外の変数の数に等しい、入出力変数以外の変数は 2 本以上の式に現れる）を書く。
- **モデル**：Anthropic の LLM（Claude）から、高性能のもの（Claude Fable 5.1。費用が高ければ Claude Opus 5.5）と、低コストのもの（Claude Haiku 5.5 か Claude Sonnet 5.5）を 1 つずつ使う。答える前に考える量の設定（effort）は、どちらのモデルも high にそろえる。呼び出しはプログラムから行ない、モデル名、指示文の版、設定、日時、使ったトークン数、応答をすべて記録する（Appendix B）。

### 2.3 判定と指標

**判定**：手法が答えた式の ID が正解集合 $R_q$（$q$ はクエリ）に含まれれば正解、含まれなければ不正解とする。データベースにない ID は不正解とし、その順位を占めるものとする。同じ ID が重なったときは、2 回目以降を除く。式どうしが等価かは判定しないので、正解式と同じ式が別の出典にあり、それを選んだ場合も不正解と数える。提案手法と baseline も同じ規則で判定する。

**指標**：正解式の数を $K = |R_q|$、手法が並べた上位 20 件を $\mathrm{Top}_{20}(q)$ として、テスト用のケースごとに

$$\mathrm{Recall@20}(q) = \frac{|R_q \cap \mathrm{Top}_{20}(q)|}{K}, \qquad \mathrm{AP@20}(q) = \frac{1}{K} \sum_{e \in R_q} P(e)$$

を求める。$P(e)$ は、正解式 $e$ が上位 20 件にあれば、$e$ の順位までの式のうち正解式の割合、なければ 0 である。AP@20 のケースの平均を MAP@20（mean average precision）とよぶ。MAP@20 は、正解式がどれだけ上位に集まっているかを表す。LLM は 20 件までしか答えないので、どの手法も上位 20 件で比べる。

**比べ方**：提案手法と baseline と同じ 10 通りの分け方を使い、分け方ごとにテスト用のケースで平均した値を求めて、10 通りの平均 ± 標準偏差と、10 通りのうち何通りで一方が上回ったかを示す。評価は、全ケースを使う場合と、合成のケースだけを使う場合（分け方はそれぞれ 10 通り）の 2 つで行なう。比べる相手の値を Table 2 に示す。

**呼び出しの回数**：LLM には解いた例を渡さないので、同じケースへの入力は、どの分け方でも同じになる。そこで、異なるテスト用のケースごとに 1 回だけ解かせ、その答えを、そのケースがテスト用に入るすべての分け方で使う。10 通りの分け方のテスト用のケースは、全ケースの評価で延べ 4,229 件、合成のケースの評価で延べ 2,432 件あり、異なるケースは合わせて 1,673 件である。したがって、呼び出しは 1 モデルにつき 1,673 回である。1 回だけ解かせるので、同じケースを解かせ直したときの答えのばらつきは測らない。

### 2.4 費用

1 回の入力は、指示文とデータベースの 952,185 トークンと、テストケースの平均 354 トークンである。データベースの部分は毎回同じなので、プロンプトキャッシュを使う。プロンプトキャッシュは、入力の先頭部分を保存しておき、次の呼び出しでは、その部分を通常の入力の 2.5〜10%（モデルによる）の料金で読み出す仕組みである（保存には、通常の入力の 1.25 倍の料金がかかる）。出力のトークン数（答える前

Table 2: Recall@20 and MAP@20 of the methods that the LLMs will be compared with (mean ± standard deviation over the 10 splits). The proposed method reranks the top 2,000 candidates of the first-stage search; the baseline ranks all the equations by the first-stage score. MAP@20 sets the precision of a correct equation outside the top 20 to zero.

| | 全ケース（1,731 件） | | 合成のケース（1,000 件） | |
|---|---|---|---|---|
| 手法 | Recall@20 | MAP@20 | Recall@20 | MAP@20 |
| baseline | 0.906 ± 0.020 | 0.680 ± 0.063 | 0.825 ± 0.024 | 0.601 ± 0.037 |
| 提案手法 | 0.981 ± 0.010 | 0.857 ± 0.055 | 0.966 ± 0.016 | 0.823 ± 0.044 |
| LLM（2 モデル） | 実行後に示す | | | |

Table 3: Estimated cost in US dollars of one run over the 1,673 test cases, assuming 2,000 output tokens per call. The second-to-last column gives the share of reading the database from the cache in the total; the last column gives the total for 70 to 8,000 output tokens per call.

| モデル | データベースの読み出し | キャッシュへの保存 | テストケースの入力 | 出力 | 合計 | 読み出しの割合 | 合計（出力 70〜8,000） |
|---|---|---|---|---|---|---|---|
| Claude Haiku 5.5 | 79.7 | 0.6 | 0.3 | 8.4 | 88.9 | 90% | 81〜114 |
| Claude Sonnet 5.5 | 159.3 | 2.4 | 1.2 | 33.5 | 196.3 | 81% | 164〜297 |
| Claude Opus 5.5 | 318.6 | 4.8 | 2.4 | 66.9 | 392.7 | 81% | 328〜593 |
| Claude Fable 5.1 | 398.3 | 11.9 | 5.9 | 167.3 | 583.4 | 68% | 422〜1,085 |

に考えた分を含む）は流してみないと分からないので、1 回 2,000 トークンと仮定した。料金は、2026 年 11 月 1 日に確かめた料金表 [2] による（Appendix C）。

見積もりを Table 3 に示す。1 モデルで 1,673 回を流す費用は 89〜583 ドルで、2 モデルでは、組み合わせによって 482〜780 ドルになる。どのモデルでも、費用の 68〜90% は、呼び出しのたびにデータベースを読み出す料金である。出力が 1 回 70〜8,000 トークンの範囲で変わると、合計は Table 3 の右端の列のように変わる。

## 3 今後の計画

- ゼミでの議論を受けて、比べ方と使う 2 つのモデルを決め、加藤先生の承認を得てから流す。
- データベースを変える作業（研究の初期に抽出した 9 報の式の抽出し直しと、LLM が作ったケースと式の人手による確認）の方針を、流す前に決める。LLM には毎回データベース全体を渡すので、データベースが変わると、もう一度費用がかかる。
- 流す前に、各モデルで数件だけ試し、答えの形と出力のトークン数を確かめる。
- 提案手法の評価と同じく、別の出典にある同じ文字列の式も正解とみなした場合の値も計算する。

## References

[1] Shota Kato and Manabu Kano. Automated physical model building from literature sources: Combining equations based on four predefined requirements. *Computers & Chemical Engineering*, 200:109147, 2025. doi:10.1016/j.compchemeng.2025.109147.

[2] Anthropic. Pricing. https://platform.claude.com/docs/en/about-claude/pricing, 2026. Accessed November 1, 2026.

## Appendix A LLM に与える指示文と入力の形

system として指示文を、user の 1 つ目のブロックとしてデータベースを、2 つ目のブロックとしてテストケースを、この順に送る。データベースのブロックの末尾にキャッシュの区切りを置くので、2 回目以降の呼び出しでは、指示文とデータベースをキャッシュから読み出す。どのモデルにも同じ指示文を使う。<TAB> はタブ文字を表す。データベースは、11,146 行のうち先頭の 1 行だけを示す。テストケースの < > の部分には、そのケースの説明文と変数記号を入れる。

```
[system]
You are an expert in physical modeling. You select equations from an equation database for building a physical model.

# Task
An engineer wants to build a physical model: a set of equations with which the output variables can be computed once the
↪  input variables are given. The engineer describes the model and lists its input variables and output variables.
↪  Choose, from the equation database, the equations that the engineer should use.
```

```
The set of equations that forms the model satisfies these conditions:
1. Every input variable and every output variable appears in at least one equation of the set.
2. The number of equations equals the number of variables of the set that are not input variables.
3. Every variable of the set that is neither an input nor an output variable appears in at least two equations of the
↪  set.

# Equation database
<equation_database> lists equations extracted from the literature, in no particular order. Each line has three fields
↪  separated by tabs: the equation ID, the domain label of the equation, and the equation in LaTeX.

# Answer
For the task in <task>, list up to 20 equation IDs, from the most to the least likely to belong to the model. Put first
↪  the equations that you judge to form the model. If they are fewer than 20, continue with the other equations most
↪  likely to be needed for this model, until the list has 20 IDs. Use only IDs from the database, and list each ID
↪  once. In the task, variable symbols in a list are separated by " ; ".

[user, block 1: equation database, cached]
<equation_database count="11146">
1<TAB>Educational Psychology Time-Series Analysis<TAB>MSSD = \frac{1}{N-1} \sum_{t=2}^{N} (x_t - x_{t-1})^2
...
</equation_database>

[user, block 2: test case]
<task>
Description: <description of the model>
Input variables: <symbol> ; <symbol> ; ...
Output variables: <symbol> ; ...
</task>

List the IDs of the database equations for this task.

[output format (JSON schema)]
{"type": "object", "properties": {"equation_ids": {"type": "array", "items": {"type": "integer"}}}, "required":
↪  ["equation_ids"], "additionalProperties": false}
```

## Appendix B　呼び出しの設定と記録

- **考える量**：effort は high とする（Claude Fable 5.1 と Claude Sonnet 5.5 では既定の値、Claude Opus 5.5 と Claude Haiku 5.5 では既定の medium より 1 段上）。答える前に考える量は、モデルが問題に応じて決める設定（adaptive thinking）にし、考えた内容の要約も受け取る。出力の無作為さを決める値（temperature）は、これらのモデルでは指定できないので既定のままである。
- **出力の上限**：32,000 トークン（考えた分を含む）。入力の最大は 952,918 トークン（最も長いテストケースのとき）で、出力の上限と合わせても 100 万トークンに収まる。出力が長くなりうるので、応答は少しずつ受け取る（ストリーミング）。
- **答えの形**：答えを、式の ID（整数）の列だけから成る決まった形に固定する機能（structured outputs）を使う（形の指定は Appendix A の最後）。読めない応答や、モデルが答えを拒んだ場合は、正解式を 1 本も挙げなかったものとして数える。答えたモデルが変わると比べられないので、拒まれたときに別のモデルへ切り替える機能は使わない。
- **キャッシュ**：保存の期間は 5 分である。各モデルの最初の 1 件を単独で流してキャッシュに保存し、残りは 8 本並列で流す。並列の呼び出しが同時に始まると、それぞれが保存の料金を払うためである。
- **予算の上限**：モデルごとに費用の上限を決め、累計が上限に達したら止める。同じコマンドで、続きから再開できる。
- **記録**：要求したモデルと応答したモデルの名前、指示文の版、effort、出力の上限、式の並びを決めた乱数の種、日時、トークン数（入力、キャッシュへの保存、キャッシュからの読み出し、出力）、応答の全文、考えた内容の要約、ケースのファイルとテストケースの文のハッシュ値を、呼び出しごとに残す。
- **テスト用のケースの内訳**：異なる 1,673 件は、合成のケース 994 件、異なる出典の式を組み合わせたケース 601 件、1 つの出典の式だけから成るケース 78 件である。1,731 件のうち 58 件は、10 通りのどの分け方でもテスト用に入らない。

## Appendix C　費用の見積もりの前提

見積もりに使った料金を Table 4 に示す。

1 モデルの費用は、呼び出しの回数を $n = 1{,}673$、指示文とデータベースのトークン数を $P = 952{,}185$、

Table 4: Prices in US dollars per million tokens (Anthropic pricing page, November 1, 2026), with the 5-minute cache. For Claude Haiku 5.5, the prices for prompts over 100,000 tokens are shown.

| モデル | 入力 | キャッシュへの保存 | キャッシュからの読み出し | 出力 |
|---|---|---|---|---|
| Claude Haiku 5.5 | 0.50 | 0.625 | 0.05 | 2.50 |
| Claude Sonnet 5.5 | 2 | 2.50 | 0.10 | 10 |
| Claude Opus 5.5 | 4 | 5 | 0.20 | 20 |
| Claude Fable 5.1 | 10 | 12.50 | 0.25 | 50 |

テストケースのトークン数の平均を $T = 354$、1 回の出力のトークン数を $O$ として、

$$n\,(P\,r_{\mathrm{read}} + T\,r_{\mathrm{in}} + O\,r_{\mathrm{out}}) + P\,r_{\mathrm{write}}$$

で見積もった。$r_{\mathrm{read}}$、$r_{\mathrm{in}}$、$r_{\mathrm{out}}$、$r_{\mathrm{write}}$ は、キャッシュからの読み出し、入力、出力、キャッシュへの保存の、1 トークンあたりの料金である。キャッシュは毎回当たり、保存はモデルごとに 1 回とした（呼び出しが 5 分以上空くと、もう 1 回保存の料金がかかる）。トークン数は、Anthropic のトークンを数える機能（無料）で、送る形のまま数えた。$T$ は、最も長いテストケース（734 トークン）のトークン数を、文字数の比で、解かせる 1,673 件の平均に換算した値である。$O$ は 2,000 トークンを中心とし、70 トークン（10 月 14 日の試しの呼び出しで、考えずに答えた応答の実測）と 8,000 トークン（長く考えた場合の想定）の範囲も示した。